VALIDACIÓN DE CALIDAD DE DATOS

1. VALIDACIÓN DE INTEGRIDAD REFERENCIAL

Verificar que todas las relaciones entre tablas se cumplan correctamente, es decir, que no existan registros “huérfanos” que rompan el modelo relacional.

Query 1: Trips con vehicle inexistente

```
SELECT COUNT(*) AS invalid_trips
  FROM trips t
  LEFT JOIN vehicles v ON t.vehicle_id = v.vehicle_id
  WHERE v.vehicle_id IS NULL;
```

Results 1

SELECT COUNT(*) AS invalid_trips FROM | Enter a SQL expression to filter results (use Ctrl+S

	123 invalid_trips
1	0

Query 2: Deliveries sin trip válido

```
SELECT COUNT(*) AS invalid_trips
  FROM trips t
  LEFT JOIN vehicles v ON t.vehicle_id = v.vehicle_id
  WHERE v.vehicle_id IS NULL;
```

Results 1

SELECT COUNT(*) AS invalid_trips FROM | Enter a SQL expression to filter results (use Ctrl

	123 invalid_trips
1	0

Query 3: Maintenance sin vehicle válido

```sql
SELECT COUNT(*) AS invalid_maintenance
FROM maintenance m
LEFT JOIN vehicles v ON m.vehicle_id = v.vehicle_id
WHERE v.vehicle_id IS NULL;
```

Results 1

SELECT COUNT(*) AS invalid_maintenanc | Enter a SQL expression to filter results (use Ct

Grid

	123 invalid_maintenance
1	0

Análisis

Estas validaciones garantizan que el modelo relacional se mantiene consistente, lo cual es fundamental en sistemas transaccionales. La integridad referencial evita inconsistencias que podrían afectar reportes, métricas operativas y procesos downstream como pipelines de datos o dashboards.

2. CONSISTENCIA TEMPORAL

Asegurar que las fechas tengan sentido lógico dentro del negocio.

Query: arrival debe ser mayor que departure

```sql
SELECT COUNT(*) AS invalid_time_records
FROM trips
WHERE arrival_datetime IS NOT NULL
AND arrival_datetime < departure_datetime;
```

Results 1

SELECT COUNT(*) AS invalid_time_record | Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	123 invalid_time_records
1	0

Query adicional (entregas)

```sql
SELECT COUNT(*) AS invalid_delivery_times
  FROM deliveries
  WHERE delivered_datetime IS NOT NULL
  AND delivered_datetime < scheduled_datetime;

select * from deliveries
```

Results 1 | deliveries 2

Enter a SQL expression to filter results (use Ctrl+Space)

	invalid_delivery_times
1	177,134

Análisis

Podemos ver la gran cantidad de inconsistencias temporales, esto rompe análisis como duración de viajes o puntualidad

3. VALIDACIÓN DE DISTRIBUCIONES ESTADÍSTICAS

3.1 Entregas por viaje

```sql
SELECT
    AVG(delivery_count) AS avg_deliveries,
    MIN(delivery_count) AS min_deliveries,
    MAX(delivery_count) AS max_deliveries
FROM (
    SELECT trip_id, COUNT(*) AS delivery_count
    FROM deliveries
    GROUP BY trip_id
) t;
```

Results 3

Enter a SQL expression to filter results (use Ctrl+Space)

	avg_deliveries	min_deliveries	max_deliveries
1	4.0020110055	2	6

Análisis

El promedio de entregas por viaje es aproximadamente **4**, lo cual coincide con la regla de negocio definida en la generación de datos. Además, los valores mínimo (2) y máximo (6) se mantienen dentro del rango esperado, lo que indica que la distribución fue correctamente controlada mediante probabilidades.

Este comportamiento demuestra que los datos no fueron generados de forma uniforme o aleatoria, sino que siguen una lógica operativa coherente con un sistema de logística de última milla, donde cada viaje agrupa múltiples entregas.

3.2 Distribución de horarios de viajes

```sql
SELECT
    EXTRACT(HOUR FROM departure_datetime) AS hour,
    COUNT(*) AS total_trips
FROM trips
GROUP BY hour
ORDER BY hour;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

	hour	total_trips
1	0	1,683
2	1	1,719

Resultado (resumen interpretado)

- Madrugada (0–5): 1700 viajes por hora
- Horario laboral (6–19): entre 5000 y 7000 viajes
- Pico máximo: entre 8:00 y 11:00 (aprox. 6900 viajes)
- Noche (20–23): baja nuevamente (1700)

Análisis

La distribución de viajes evidencia un comportamiento claramente alineado con la operación logística real. Se observa un incremento significativo en horas laborales (6:00–19:00), con picos en la mañana (8:00–11:00), lo cual es consistente con ventanas de despacho y planificación de entregas.

3.3 Validación de capacidad de vehículos

```sql
SELECT COUNT(*) AS overload_trips
FROM trips t
JOIN vehicles v ON t.vehicle_id = v.vehicle_id
WHERE t.total_weight_kg > v.capacity_kg;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

	overload_trips
1	0

Análisis

No se identificaron viajes que excedan la capacidad máxima de los vehículos, lo que confirma que la lógica de generación de datos respeta las restricciones físicas del sistema logístico.

Este control es crítico, ya que inconsistencias en este aspecto invalidará cualquier análisis relacionado con eficiencia operativa, costos de transporte o planificación de carga.

4. VALIDACIÓN FINAL DE VOLUMEN DE DATOS

Evidencia 1: Resultado desde archivo de resumen


```
{
  "generation_date": "2026-03-24T17:19:18.676734",
  "total_records": 505569,
  "table_counts": {
    "vehicles": 200,
    "drivers": 400,
    "routes": 50,
    "trips": 100000,
    "deliveries": 400001,
    "maintenance": 4920
  },
  "validations_passed": true
}
```


Evidencia 2: Validación directa en base de datos

```sql
SELECT 'vehicles' AS table_name, COUNT(*) FROM vehicles
UNION ALL
SELECT 'drivers', COUNT(*) FROM drivers
UNION ALL
SELECT 'routes', COUNT(*) FROM routes
UNION ALL
SELECT 'trips', COUNT(*) FROM trips
UNION ALL
SELECT 'deliveries', COUNT(*) FROM deliveries
UNION ALL
SELECT 'maintenance', COUNT(*) FROM maintenance;
```

Results 1 | trips 2 | vehicles 3 | Results 4

Enter a SQL expression to filter results (use Ctrl+Space)

	table_name	count
1	vehicles	200
2	drivers	400
3	routes	48
4	trips	100,000
5	deliveries	400,001
6	maintenance	4,920

En términos generales, los volúmenes generados son altamente consistentes con los requerimientos del negocio, alcanzando un total de más de 500,000 registros. Sin embargo, se identifican ligeras variaciones en algunas tablas.

En la tabla routes, se generaron 48 registros en lugar de los 50 esperados, lo cual puede deberse a la lógica de generación o cortes anticipados en el loop. Por otro lado, la tabla deliveries presenta un registro adicional, lo cual es consistente con la lógica iterativa que detiene la generación cuando se supera el umbral definido.

Finalmente, la tabla maintenance contiene 4,920 registros en lugar de 5,000, lo cual se explica por la dependencia directa del número de viajes por vehículo, ya que los mantenimientos se generan aproximadamente cada 20 viajes. Esto indica que la generación está basada en reglas de negocio y no en valores fijos arbitrarios.